

Exemplo didático – Derivação de uma declaração de variável em Java

1. Linguagem escolhida e fonte da gramática

Linguagem escolhida: Java

Fonte de referência: Java Language Specification (JLS), seção 14.4 – *Local Variable Declarations*, da documentação oficial da Oracle.

Fonte consultada: [Java Language Specification – Chapter 14, seção 14.4](#)

A gramática apresentada pela especificação do Java utiliza uma notação baseada em produções gramaticais. Para esta atividade, foi selecionado apenas o subconjunto necessário para representar uma declaração simples de variável.

A especificação define, entre outras, as produções `LocalVariableDeclaration`, `LocalVariableType`, `VariableDeclaratorList`, `VariableDeclarator` e `VariableInitializer`, que descrevem a estrutura de uma declaração de variável local em Java.

2. Código gerado

O trecho de código escolhido para a derivação é:

```
int idade = 20;
```

Esse código declara uma variável chamada `idade`, do tipo `int`, e atribui inicialmente o valor 20.

3. Regras gramaticais utilizadas

Para concentrar a derivação na declaração de uma variável, foi utilizada uma versão reduzida das regras presentes na especificação do Java.

Os símbolos entre `<` `>` representam **não terminais**. Os elementos que aparecem diretamente no código representam **terminais**.

`<declaracao> ::= <tipo> <lista_declaradores> ;`

`<tipo> ::= int`

`<lista_declaradores> ::= <declarador>`

`<declarador> ::= <identificador> = <inicializador>`

`<identificador> ::= idade`

`<inicializador> ::= <numero>`

`<numero> ::= 20`

Significado das principais regras

- `<declaracao>` representa uma declaração de variável completa.
- `<tipo>` representa o tipo da variável. Neste exemplo, será `int`.
- `<lista_declaradores>` representa a lista de variáveis que estão sendo declaradas.
- `<declarador>` representa o nome da variável e, neste caso, sua inicialização.
- `<identificador>` representa o nome da variável, que será `idade`.
- `<inicializador>` representa o valor atribuído à variável.
- `<numero>` representa o valor inteiro utilizado na inicialização.

A especificação oficial do Java define uma declaração de variável local como uma combinação de tipo e declarador, podendo ainda conter uma inicialização. As regras oficiais são mais gerais que as utilizadas aqui; elas foram reduzidas para facilitar a demonstração da derivação.

4. Derivação

A derivação começa pelo símbolo não terminal `<declaracao>` e, em cada etapa, um não terminal é substituído de acordo com uma das produções escolhidas.

`<declaracao>`

$\Rightarrow$ `<tipo> <lista_declaradores> ;`

$\Rightarrow$ `int <lista_declaradores> ;`

$\Rightarrow$ `int <declarador> ;`

$\Rightarrow$ `int <identificador> = <inicializador> ;`

$\Rightarrow$ `int idade = <inicializador> ;`

$\Rightarrow$ `int idade = <numero> ;`

$\Rightarrow$ `int idade = 20 ;`

Portanto, a sequência final de terminais é:

```
int idade = 20 ;
```

Na forma concreta da linguagem Java:

```
int idade = 20;
```

5. Terminais e não terminais

Na gramática utilizada, os **não terminais** são os símbolos que precisam ser substituídos durante a derivação. Eles são representados entre < >.

Não terminais

```
<declaracao>  
<tipo>  
<lista_declaradores>  
<declarador>  
<identificador>  
<inicializador>  
<numero>
```

Terminais

Os terminais são os símbolos que permanecem na sentença final:

```
int  
idade  
=  
20  
;
```

Eles são chamados de terminais porque não são mais substituídos por outras regras durante a derivação.

6. Resultado e explicação

A partir do símbolo inicial <declaracao>, foram aplicadas sucessivamente as produções selecionadas. Primeiro, a regra de declaração foi utilizada para separar o tipo, o declarador e o ponto e vírgula. Em seguida, <tipo> foi substituído por `int`.

Depois, <declarador> foi expandido para representar o identificador, o operador de atribuição e o inicializador. O identificador <identificador> foi substituído por `idade`, enquanto <inicializador> foi substituído por <numero>, que finalmente foi substituído pelo terminal `20`.

Dessa maneira, a derivação chegou à sequência:

```
int idade = 20 ;
```

que corresponde ao código válido em Java:

```
int idade = 20;
```

A derivação demonstra que o código não foi escolhido de maneira arbitrária: ele pode ser obtido por meio de substituições sucessivas a partir do símbolo inicial e das regras gramaticais selecionadas.